

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO



FACULTAD DE CIENCIAS DE LA COMPUTACIÓN Y DISEÑO DIGITAL CARRERA DE INGENIERÍA EN SOFTWARE

TEMA:

GA – EXPOSICIÓN SEGUNDO CORTE: HERRAMIENTAS CASE (VISUAL PARADIGM)

MATERIA:

INGENIERÍA EN REQUERIMIENTOS

DOCENTE:

ING. GUERRERO ULLOA GLEISTON CICERÓN, PhD.

INTEGRANTES:

FUERTES ARRAES EDSON DANIEL, CI: 0929115087, Correo: efuertes2@uteq.edu.ec

CEDEÑO AVILA WINSTON DAMIAN, CI: 0942833492, Correo: wcedenoa2@uteq.edu.ec

CEDEÑO CORONADO WILSON LIZANDRO, CI: 1206867200, Correo: wcedenoc2@uteq.edu.ec

EQUIPO G

CURSO:

CUARTO SEMESTRE INGENIERÍA EN SOFTWARE “B”

SISTEMA PROYECTO FIN DE CURSO:

MUNDIPETS (DANIEL FUERTES - TITULAR PFC))

Repositorio GitHub:

<https://github.com/MiloSaurio4kHD/PFC-MundiPets-MDD-EquipoG.git>

QUEVEDO – LOS RÍOS – ECUADOR

2026 – 2027 PPA

Índice

1.	Resumen	4
2.	Marco teórico	4
2.1.	Ingeniería dirigida por modelos (MDE)	4
2.2.	Model-Driven Architecture (MDA): CIM, PIM y PSM	4
2.3.	Desarrollo dirigido por modelos (MDD)	4
2.4.	Transformaciones M2M y M2T	5
2.5.	Metamodelos, perfiles UML, estereotipos y plantillas	5
2.6.	Forward, reverse y round-trip engineering	5
3.	Justificación de la herramienta MDD seleccionada	6
4.	Descripción del subsistema del PFC modelado	6
4.1.	Subsistema seleccionado: gestión del perfil de mascota y validación de información médica	6
4.2.	Actores involucrados	6
4.3.	Requisitos funcionales relevantes	6
4.4.	Alcance incluido y alcance excluido	7
4.5.	Clases de dominio candidatas para el modelo de origen	7
4.6.	Representatividad para la demostración MDD	7
5.	Modelo UML de origen	7
5.1.	Diagrama de clases	7
5.2.	Diagramas complementarios (si aportan)	7
5.3.	Perfiles y estereotipos aplicados	7
6.	Configuración del mecanismo de generación	7
7.	Proceso de generación de código	7
8.	Verificación del código generado	7
9.	Cierre del ciclo (roundtrip) y trazabilidad	7
10.	Reparto del trabajo por integrante	7
11.	Conclusiones	7
12.	Referencias	8
	Anexos	8

Índice de figuras

Índice de tablas

1.	Niveles de abstracción de MDA aplicados a MundiPets.	4
2.	Comparación entre transformaciones M2M y M2T.	5
3.	Actores del subsistema de perfil de mascota y validación médica.	6
4.	Requisitos funcionales núcleo del subsistema.	6
5.	Delimitación del alcance de la demostración.	7

1 Resumen

2 Marco teórico

El desarrollo dirigido por modelos reúne un conjunto de enfoques de ingeniería de software en los que los modelos dejan de ser únicamente documentación y pasan a convertirse en artefactos centrales del proceso de construcción. La literatura reciente destaca que el valor práctico de estos enfoques aparece cuando los modelos se vinculan con mecanismos de validación, transformación y generación automática, de forma que sea posible conservar trazabilidad entre decisiones de diseño e implementación [1-3]. En el contexto de MundiPets, este principio permite partir de modelos UML del dominio y utilizarlos como entrada controlada para obtener una base de código reproducible.

2.1 Ingeniería dirigida por modelos (MDE)

Model-Driven Engineering (MDE) es el enfoque más amplio. Considera los modelos, metamodelos y transformaciones como elementos de primera clase durante distintas actividades del ciclo de vida. Su objetivo no se limita a generar código: también puede apoyar análisis, validación, consistencia, interoperabilidad, evolución y trazabilidad. En la práctica, la automatización es una característica clave, porque reduce la repetición de tareas mecánicas y permite trasladar información entre niveles de abstracción de manera sistemática [2-4].

Aplicado a MundiPets, la MDE comprende la cadena completa que conecta los requisitos del sistema con sus modelos de análisis y diseño, el diagrama de clases construido en Visual Paradigm y los artefactos de implementación derivados. Por ejemplo, una entidad del dominio como *Mascota* puede mantenerse relacionada con atributos, asociaciones y operaciones modeladas, de modo que los cambios estructurales puedan rastrearse hasta el código que corresponda.

2.2 Model-Driven Architecture (MDA): CIM, PIM y PSM

Model-Driven Architecture (MDA) es una forma estructurada de aplicar principios dirigidos por modelos mediante la separación de niveles de abstracción. Estudios recientes sobre MDA mantienen la distinción entre Computation Independent Model (CIM), Platform Independent Model (PIM) y Platform Specific Model (PSM), y destacan que las transformaciones entre dichos niveles permiten reducir dependencias tempranas de la tecnología y automatizar parte del paso hacia la implementación [4, 5].

Tabla 1: Niveles de abstracción de MDA aplicados a MundiPets.

Nivel	Propósito	Aplicación a MundiPets
CIM	Describe el problema, objetivos, procesos y necesidades del negocio sin decidir tecnología.	Necesidad de centralizar perfiles de mascotas, adopción/cruza responsable y validación de información sanitaria; requisitos y actores del SRS.
PIM	Describe estructura y comportamiento del software sin comprometerlo con una plataforma concreta.	Modelo UML del subsistema con clases como <i>Mascota</i> , <i>HistorialMedico</i> , <i>DocumentoVeterinario</i> , <i>CarnetVacunacion</i> y <i>ValidacionMedica</i> , con asociaciones y operaciones.
PSM	Añade decisiones tecnológicas necesarias para construir el sistema en una plataforma objetivo.	El mismo modelo enriquecido con convenciones del lenguaje, tipos concretos, paquetes, persistencia y estereotipos/anotaciones según la configuración definitiva del generador.

Para la demostración del equipo, el PIM debe conservar la lógica del dominio de MundiPets sin depender del lenguaje de salida. El PSM se obtiene cuando el modelo incorpora información específica para la generación. Esta separación es importante porque permite explicar con claridad qué información pertenece al problema y cuál fue introducida exclusivamente para materializar el código [5].

2.3 Desarrollo dirigido por modelos (MDD)

Model-Driven Development (MDD) se concentra específicamente en utilizar modelos como principal insumo del desarrollo y en automatizar la obtención de artefactos ejecutables o cercanos a la implementación. En este sentido, puede entenderse como una aplicación orientada al desarrollo dentro del marco más amplio de MDE. La generación automática de código es uno de sus mecanismos más visibles y ha sido estudiada como medio para aumentar productividad, repetibilidad y trazabilidad, aunque su efectividad depende de la calidad del modelo y del generador utilizado [1, 4].

En MundiPets, la demostración MDD no pretende generar la plataforma completa. El propósito es evidenciar un ciclo controlado sobre un subsistema representativo: modelar sus clases en Visual Paradigm, aplicar la configuración de generación, obtener código, verificar la correspondencia modelo-código y ejecutar un cambio que demuestre el roundtrip.

2.4 Transformaciones M2M y M2T

Las transformaciones Model-to-Model (M2M) convierten un modelo origen en otro modelo destino. Dentro del ecosistema OMG, QVT (Query/View/Transformation) constituye la familia de lenguajes de referencia para expresar transformaciones entre modelos conformes a metamodelos. Conceptualmente, una transformación M2M puede utilizarse para refinar un PIM hacia un PSM, incorporando información tecnológica que no estaba presente en el modelo independiente de plataforma [5].

Las transformaciones Model-to-Text (M2T) producen artefactos textuales a partir de un modelo. El caso típico es la generación de código fuente, aunque también pueden generarse archivos de configuración o documentación. MOFM2T representa la especificación OMG para este tipo de transformación basada en plantillas. Investigaciones recientes sobre M2T muestran que las plantillas combinan texto fijo con expresiones que consultan el modelo, y que la regeneración requiere mecanismos para evitar la pérdida de código escrito manualmente, como regiones protegidas o estrategias de sincronización [6].

Tabla 2: Comparación entre transformaciones M2M y M2T.

Tipo	Entrada	Salida	Ejemplo en MundiPets
M2M	Modelo UML/PIM	Modelo refinado/PSM	Agregar detalles tecnológicos, mapeos de persistencia o convenciones necesarias para el generador.
M2T	Modelo/PSM	Código o texto	Generar clases, atributos, métodos y otros archivos del proyecto a partir del modelo configurado en Visual Paradigm.

2.5 Metamodelos, perfiles UML, estereotipos y plantillas

Un metamodelo define los conceptos válidos de un lenguaje de modelado y las relaciones que pueden existir entre ellos; por ello, un modelo UML debe ajustarse a las reglas de su metamodelo. Sobre esta base, los perfiles UML permiten especializar el lenguaje sin crear una notación completamente nueva. Un perfil puede introducir estereotipos, valores etiquetados y restricciones para expresar información adicional que será interpretada por herramientas o generadores.

En una demostración de MundiPets pueden emplearse, cuando la configuración elegida lo requiera, estereotipos como “Entity”, “Service” o “Controller” para diferenciar responsabilidades. Estos estereotipos no deben añadirse como decoración: su utilidad radica en que exista una regla explícita de mapeo hacia el código generado. De manera similar, las plantillas de generación establecen cómo cada elemento del modelo se convierte en texto. La automatización es reproducible únicamente cuando modelo, reglas y plantillas se versionan de forma conjunta [1-3].

2.6 Forward, reverse y round-trip engineering

Forward engineering es el proceso de derivar código u otros artefactos de implementación a partir del modelo. Reverse engineering realiza el recorrido inverso: analiza una implementación existente para reconstruir o actualizar una representación de diseño. Round-trip engineering busca mantener sincronizados ambos lados cuando existen cambios sucesivos. Este último escenario es el más exigente porque una regeneración puede sobrescribir modificaciones manuales si la herramienta no dispone de una política clara de fusión o protección [6].

Para la evidencia de MundiPets, el ciclo mínimo debe mostrar: modelo inicial → generación de código → verificación → cambio no trivial en el modelo → regeneración → comprobación del cambio. La trazabilidad entre clases, atributos, operaciones y archivos generados permite demostrar que el resultado no es una copia manual, sino una transformación repetible y verificable.

3 Justificación de la herramienta MDD seleccionada

4 Descripción del subsistema del PFC modelado

4.1 Subsistema seleccionado: gestión del perfil de mascota y validación de información médica

Para la demostración del ciclo MDD se propone delimitar el trabajo al subsistema de gestión del perfil de mascota y validación de información médica. La elección es coherente con el alcance de MundiPets, plataforma orientada a centralizar perfiles de mascotas y apoyar procesos de adopción y cruce responsable. La documentación previa del PFC del equipo (entrega parcial del ERS/SRS de MundiPets) identifica como problemas del proceso actual la publicación de información incompleta y la falta de datos médicos verificables, por lo que este subsistema concentra información estructural, relaciones entre entidades y operaciones que resultan adecuadas para una demostración de generación de código.

El subsistema es representativo porque combina operaciones de registro, actualización, consulta y validación; además, contiene asociaciones uno-a-uno y uno-a-muchos que permiten observar cómo Visual Paradigm traslada estructuras UML a construcciones del lenguaje objetivo. No se pretende modelar toda la red social, sino aislar una unidad coherente que pueda ser generada, compilada y modificada durante la demostración.

4.2 Actores involucrados

Tabla 3: Actores del subsistema de perfil de mascota y validación médica.

Actor	Participación en el subsistema	Responsabilidad principal
Propietario de mascota	Actor principal de gestión	Registra la mascota, mantiene sus datos e incorpora antecedentes o documentos permitidos.
Refugio	Gestor alternativo del perfil	Puede registrar y mantener perfiles de mascotas bajo su cuidado cuando corresponda al proceso de adopción.
Veterinaria / profesional autorizado	Actor de validación	Revisa información y documentación sanitaria y registra el estado de validación.
Adoptante o interesado en cruce	Actor de consulta	Consulta la información autorizada del perfil antes de tomar decisiones dentro de un proceso.

4.3 Requisitos funcionales relevantes

La delimitación se apoya en los requisitos ya documentados y refinados en el ERS/SRS del equipo y en la práctica de modelado UML previa del PFC. Para evitar ampliar innecesariamente el alcance, se consideran como núcleo los siguientes requisitos:

Tabla 4: Requisitos funcionales núcleo del subsistema.

ID	Requisito	Relación con el subsistema
RF-01	Registrar mascota	Crea la entidad principal y sus datos generales.
RF-02a / RF-02b	Registrar y actualizar historial médico	Mantiene antecedentes clínicos vinculados con la mascota.
RF-03	Consultar perfil completo de una mascota	Recupera la información autorizada del perfil y sus relaciones.
RF-09	Validar la información médica de una mascota	Introduce el proceso de revisión por un actor veterinario autorizado.
RF-14	Gestionar carnet de vacunación	Modela información sanitaria estructurada y su relación con vacunas y controles.
RF-16	Gestionar antecedentes genéticos y parentesco	Añade antecedentes útiles para procesos responsables de cruce.
RF-11	Administrar la privacidad de la información	Se considera requisito transversal para controlar qué datos médicos pueden ser consultados.

La práctica experimental de modelado UML del equipo relaciona estos requisitos con clases del dominio como Mascota, HistorialMedico, DocumentoVeterinario, CarnetVacunacion y otras entidades asociadas, confirmando que el subsistema dispone de suficiente riqueza estructural para construir un diagrama de clases de seis o más clases sin incorporar elementos ajenos al objetivo de la demostración.

4.4 Alcance incluido y alcance excluido

Tabla 5: Delimitación del alcance de la demostración.

Incluido en el subsistema	Fuera del alcance de esta demostración
Registro y actualización de los datos esenciales de la mascota.	Gestión completa de solicitudes de adopción.
Asociación del perfil con historial médico y documentos veterinarios.	Gestión completa de solicitudes de cruce.
Carnet de vacunación y elementos sanitarios relacionados.	Mensajería y conversaciones entre usuarios.
Validación de información/documentación médica.	Motor de inteligencia artificial y algoritmos de compatibilidad.
Antecedentes genéticos y reglas básicas de visibilidad del perfil.	Moderación global, reportes y administración integral de la red social.

4.5 Clases de dominio candidatas para el modelo de origen

Sin sustituir la responsabilidad del Integrante 1 sobre el diagrama definitivo, la documentación previa permite proponer como conjunto coherente de clases para este subsistema: Mascota, Propietario/Usuario, HistorialMedico, DocumentoVeterinario, CarnetVacunacion, Vacuna, ValidacionMedica y AntecedenteGenetico. Este conjunto supera el mínimo de seis clases exigido por la guía y permite representar cardinalidades, colecciones y operaciones con firmas completas. La denominación final debe coincidir exactamente con el archivo .vpp y con el código que se genere.

4.6 Representatividad para la demostración MDD

El subsistema seleccionado ofrece una demostración clara del enfoque MDD porque la entidad Mascota funciona como núcleo y se relaciona con objetos que poseen ciclos y responsabilidades distintas. Un cambio como agregar un atributo al perfil, una operación al historial o una relación con una entidad sanitaria puede observarse primero en UML y luego en el código regenerado. Esto facilita construir la tabla de correspondencia modelo → código y mostrar trazabilidad.

Además, la separación del subsistema evita que la exposición se convierta en una revisión general de MundiPets. El objetivo de la demostración debe permanecer en Visual Paradigm y en el recorrido modelo → generación → verificación → roundtrip, utilizando el dominio de MundiPets únicamente como caso real sobre el cual se evidencia el funcionamiento de la herramienta.

5 Modelo UML de origen

5.1 Diagrama de clases

5.2 Diagramas complementarios (si aportan)

5.3 Perfiles y estereotipos aplicados

6 Configuración del mecanismo de generación

7 Proceso de generación de código

8 Verificación del código generado

9 Cierre del ciclo (roundtrip) y trazabilidad

10 Reparto del trabajo por integrante

11 Conclusiones

El enfoque MDD aporta valor al PFC MundiPets cuando el modelo se utiliza como un artefacto operativo y no únicamente como documentación. La posibilidad de generar una base de código desde un diagrama UML reduce tareas repetitivas, favorece la consistencia estructural y crea una relación verificable entre las decisiones del modelo y los archivos producidos. La literatura reciente señala que la automatización y la trazabilidad son dos de los beneficios más relevantes de los enfoques dirigidos por modelos, siempre que exista disciplina en la gestión de modelos, reglas y artefactos generados [1, 3, 4].

En MundiPets, delimitar la demostración al subsistema de perfil de mascota y validación médica permite comprobar el ciclo sobre una parte real y representativa del PFC sin intentar automatizar el sistema completo. Este subsistema contiene entidades, asociaciones y operaciones suficientes para evidenciar generación, correspondencia modelo-código y un cambio de roundtrip. Por ello, el código generado puede ser aprovechable como línea base estructural para el desarrollo posterior, pero no debe asumirse como una implementación funcional completa: reglas de negocio, seguridad, validaciones específicas, integración y pruebas continuarán requiriendo trabajo de desarrollo.

El principal aprendizaje metodológico es que la calidad de la generación depende directamente de la precisión del modelo y de la configuración del generador. Estereotipos, multiplicidades, tipos y convenciones deben definirse con intención, porque una inconsistencia del modelo se propaga a los artefactos generados. Del mismo modo, la regeneración exige una política explícita para proteger o separar el código escrito manualmente, ya que el roundtrip es uno de los puntos donde pueden aparecer pérdidas de cambios o divergencias entre modelo e implementación [6].

12 Referencias

- [1] G. Sebastián, J. A. Gallud y R. Tesoriero, “Code generation using model driven architecture: A systematic mapping study,” *Journal of Computer Languages*, vol. 56, pág. 100 935, 2020. DOI: 10.1016/j.cola.2019.100935
- [2] D. Di Ruscio, D. Kolovos, J. de Lara, A. Pierantonio, M. Tisi y M. Wimmer, “Low-code development and model-driven engineering: Two sides of the same coin?” *Software and Systems Modeling*, vol. 21, n.º 2, págs. 437-446, 2022. DOI: 10.1007/s10270-021-00970-2
- [3] I. David, K. Aslam, I. Malavolta y P. Lago, “Collaborative Model-Driven Software Engineering — A systematic survey of practices and needs in industry,” *Journal of Systems and Software*, vol. 199, pág. 111 626, 2023. DOI: 10.1016/j.jss.2023.111626
- [4] M. R. Krouwel, M. Op ’t Land y H. A. Proper, “From enterprise models to low-code applications: mapping DEMO to Mendix; illustrated in the social housing domain,” *Software and Systems Modeling*, vol. 23, págs. 837-864, 2024. DOI: 10.1007/s10270-024-01156-2
- [5] N. Silega, M. Noguera, Y. I. Rogozov, V. S. Lapshin y T. González, “Transformation From CIM to PIM: A Systematic Mapping,” *IEEE Access*, vol. 10, págs. 90 857-90 872, 2022. DOI: 10.1109/ACCESS.2022.3201556
- [6] S. Almutairi, A. Zolotas y D. Kolovos, “Towards Round-Trip Engineering of Code Fragments Embedded in Models,” en *ACM/IEEE 25th International Conference on Model Driven Engineering Languages and Systems Companion (MODELS ’22 Companion)*, 2022, págs. 529-538. DOI: 10.1145/3550356.3561578

Anexos

Estructura del repositorio GitHub

```
PFC-MundiPets-MDD-EquipoG/  
  README.md  
  docs/  
    informe.tex  
    informe.pdf  
    referencias.bib  
    figuras/  
  modelo/  
    <proyecto_nativo>.vpp  
  perfiles/  
  plantillas/  
  generado/  
  evidencias/  
    capturas/  
    video-demo.mp4  
  exposicion/  
    diapositivas.pdf
```

Anexos adicionales

Tabla de mapeo extensa (modelo → código)

Listados de código representativo (fragmentos)

Instrucciones detalladas de reproducibilidad